Universidad politécnica de victoria

Ingeniería mecatrónica

Clasificación multiclase con datos tabulares

Sistemas de visión artificial

Profesor

M.C. Carlos Antonio Tovar Garcia

Alumna

Itzelt Yamileth Cedillo Hernandez 2330457

Marzo 2023

Objetivo

Entrenar y evaluar 3 modelos distintos de clasificacion multiclase usando datasets tabulares (filas = ejemplos, columnas = variables), aplicando preprocesamiento correcto, metricas adecuadas y analisis de errores.

A) Descripcion de los datasets

1.1 Modelo 1 . Iris Dataset

El primer modelo trabaja con el dataset Iris (Kaggle: uciml/iris), que es uno de los datasets mas conocidos y usados en aprendizaje automatico, contiene mediciones morfologicas de flores de tres especies del genero Iris (setosa, versicolor y virginica), cada fila representa un ejemplar y cada columna una caracteristica como largo y ancho de sepalo y petalo, se usa mucho como benchmark para problemas de clasificacion multiclase porque es facil de entender, relativamente pequeño y sus clases son linealmente separables, lo que permite probar algoritmos y arquitecturas de manera sencilla y comparar desempeno entre modelos.

Cuadro 1: Caracteristicas del dataset — Iris

Parametro	Valor
Muestras	150
Variables de entrada	4 (100% numericas, sin valores nulos)
Numero de clases	3
Tipo de variables	Numericas continuas (morfometria floral)

Cuadro 2: Distribucion de clases — Iris

Clase	Indice	% Aprox.
Iris-setosa	0	33.3 %
Iris-versicolor	1	33.3 %
Iris-virginica	2	33.3 %

El dataset esta perfectamente balanceado con 50 muestras por clase. La clase Iris-setosa es facilmente separable de las otras dos, mientras que Versicolor y Virginica presentan cierto solapamiento en el espacio de caracteristicas.

1.2 Modelo 2 . Glass Identification Dataset

El segundo modelo usa el dataset Glass Identification (Kaggle: uciml/glass), que contiene analisis quimicos de distintos tipos de vidrio obtenidos de fragmentos recogidos en escenas de crimen, cada fila es un fragmento y cada columna representa la cantidad de distintos oxidos metalicos presentes, el objetivo es clasificar el tipo de vidrio segun su composicion quimica, es un dataset mas dificil que Iris porque tiene 6 clases desbalanceadas y algunas composiciones muy similares entre clases, lo que lo hace ideal para probar la capacidad del modelo de distinguir casos con alta similitud y poco numero de muestras.

Cuadro 3: Caracteristicas del dataset — Glass Identification

Parametro	Valor
Muestras	214
Variables de entrada	9 (composicion de oxidos metalicos)
Numero de clases	6
Tipo de variables	Numericas continuas

Cuadro 4: Distribucion de clases — Glass Identification

Clase	Indice	%
Building Windows Float	0	32.7 %
Building Windows Non-Float	1	35.5 %
Vehicle Windows Float	2	13.6 %
Containers	3	8.9 %
Tableware	4	4.2 %
Headlamps	5	5.1 %

El dataset presenta desbalance significativo: las clases Building Windows dominan con ~68%, mientras que Tableware representa solo el 4.2%. Se aplica class_weight para compensar. Los valores originales de la etiqueta (1,2,3,5,6,7) fueron remapeados a indices (0,1,2,3,4,5).

1.3 Modelo 3 .Wine Dataset (UCI)

El tercer modelo usa el dataset Wine (Kaggle: uciml/wine-data-set), que contiene analisis quimicos de vinos cultivados en la misma region de Italia pero provenientes de tres vinicultores distintos, cada fila representa un vino y cada columna un componente quimico

como flavonoides, prolina o intensidad de color, el objetivo es predecir el origen del vino a partir de estos 13 constituyentes, es un dataset ideal para probar como la composicion quimica puede distinguir claramente entre clases aunque los rangos de las variables sean amplios.

Cuadro 5: Caracteristicas del dataset — Wine UCI

Parametro	Valor
Muestras	178
Variables de entrada	13 (constituyentes quimicos)
Numero de clases	3
Tipo de variables	Numericas continuas

Cuadro 6: Distribucion de clases — Wine UCI

Clase	Indice	%
class_1	0	33.1 %

Clase	Indice	%
class_2	1	39.9 %
class_3	2	26.9 %

Leve desbalance: class_2 domina con ~40% y class_3 es la mas escasa (~27%). Los constituyentes quimicos (Flavanoids, Proline, Color intensity) tienen alto poder discriminativo entre origenes.

B) Descripcion del preprocesamiento aplicado

Preprocesamiento común a los tres modelos

División del dataset

se dividio el dataset en 70% entrenamiento, 15% validacion y 15% prueba, usando stratify=y para mantener la proporcion de clases y SEED=42 para que los resultados sean reproducibles

Escalamiento

Se aplico StandardScaler ajustandolo solo con X_train y despues se transformo X_val y X_test para evitar fuga de informacion

Class weights

Se calcularon con compute_class_weight('balanced') y se pasaron al fit() para compensar el desbalance entre clases

One hot encoding

Se uso LabelEncoder y luego to_categorical para obtener la representacion one hot necesaria para categorical_crossentropy.

Preprocesamiento especifico por modelo

Cuadro 7: Encoding adicional por modelo

Modelo	Encoding adicional
Iris	LabelEncoder sobre columna Species. Sin columnas ID. Dataset limpio sin valores nulos.
Glass	Eliminacion de columna Id. LabelEncoder sobre columna Type. Remapeo (1,2,3,5,6,7) -> (0,1,2,3,4,5).
Wine	LabelEncoder sobre columna class. Valores originales (1,2,3) -> indices (0,1,2).

C) Descripción de los Modelos

Los tres modelos son redes neuronales densas (MLP) implementadas con TensorFlow/Keras, todas usan categorical_crossentropy como funcion de perdida, optimizador Adam con

lr=0.001, regularizacion L2 de 1e-4, EarlyStopping con restore_best_weights=True y activacion softmax en la capa de salida para clasificacion multiclase.

3.1 Modelo 1 .Iris , 2 capas ocultas.

```
def create_model_1(input_dim, num_classes):
    inputs = tf.keras.Input(shape=(input_dim,))
    x = tf.keras.layers.Dense(
        16, activation='relu',
        kernel_regularizer=tf.keras.regularizers.l2(1e-4))(inputs)
    x = tf.keras.layers.Dropout(0.15)(x)
    x = tf.keras.layers.Dense(
        8, activation='relu',
        kernel_regularizer=tf.keras.regularizers.l2(1e-4))(x)
    x = tf.keras.layers.Dropout(0.10)(x)
    outputs = tf.keras.layers.Dense(num_classes, activation='softmax')(x)
    return tf.keras.Model(inputs, outputs)
```

Listing 1: Arquitectura del Modelo 1 — Iris

Cuadro 8: Hiperparametros — Modelo 1

Parametro	Valor
Capas ocultas	2

Parametro	Valor
Neuronas por capa	16 -> 8
Activaciones ocultas	ReLU
Activacion de salida	Softmax (3 neuronas)
Dropout	0.15 -> 0.10
BatchNormalization	No
Regularizacion L2	Si (1e-4)
Optimizador	Adam (lr = 0.001)
Funcion de perdida	categorical_crossentropy
Metrica de entrenamiento	CategoricalAccuracy, AUC-ROC
Batch size	16
Epocas maximas	300
Early Stopping	patience=30, monitor=val_loss

3.2 Modelo 2 — Glass Identification , 3 capas ocultas.

```
def create_model_2(input_dim, num_classes):
```

```
    inputs = tf.keras.Input(shape=(input_dim,))
    x = tf.keras.layers.Dense(
        64, activation='relu',
        kernel_regularizer=tf.keras.regularizers.l2(1e-4))(inputs)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Dropout(0.30)(x)
    x = tf.keras.layers.Dense(
        32, activation='relu',
        kernel_regularizer=tf.keras.regularizers.l2(1e-4))(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Dropout(0.20)(x)
    x = tf.keras.layers.Dense(
        16, activation='relu',
        kernel_regularizer=tf.keras.regularizers.l2(1e-4))(x)
    x = tf.keras.layers.Dropout(0.10)(x)
    outputs = tf.keras.layers.Dense(num_classes, activation='softmax')(x)
    return tf.keras.Model(inputs, outputs)
```

Listing 2: Arquitectura del Modelo 2 — Glass Identification

Cuadro 9: Hiperparametros — Modelo 2

Parametro	Valor
Capas ocultas	3

Parametro	Valor
Neuronas por capa	64 -> 32 -> 16
Activaciones ocultas	ReLU
Activacion de salida	Softmax (6 neuronas)
Dropout	0.30 -> 0.20 -> 0.10
BatchNormalization	Si (capas 1 y 2)
Regularizacion L2	Si (1e-4)
Optimizador	Adam (lr = 0.001)
Funcion de perdida	categorical_crossentropy
Metrica de entrenamiento	CategoricalAccuracy, AUC-ROC
Batch size	16
Epocas maximas	300
Early Stopping	patience=25, monitor=val_loss

3.3 Modelo 3 — Wine UCI ,4 capas ocultas.

```
def create_model_3(input_dim, num_classes):

    inputs = tf.keras.Input(shape=(input_dim,))

    x = tf.keras.layers.Dense(
```

```
        128, activation='tanh',   # tanh en primera capa
        kernel_regularizer=tf.keras.regularizers.l2(1e-4))(inputs)
    x = tf.keras.layers.Dropout(0.30)(x)
    x = tf.keras.layers.Dense(
        64, activation='relu',
        kernel_regularizer=tf.keras.regularizers.l2(1e-4))(x)
    x = tf.keras.layers.Dropout(0.20)(x)
    x = tf.keras.layers.Dense(
        32, activation='relu',
        kernel_regularizer=tf.keras.regularizers.l2(1e-4))(x)
    x = tf.keras.layers.Dropout(0.15)(x)
    x = tf.keras.layers.Dense(
        16, activation='relu',
        kernel_regularizer=tf.keras.regularizers.l2(1e-4))(x)
    x = tf.keras.layers.Dropout(0.10)(x)
    outputs = tf.keras.layers.Dense(num_classes, activation='softmax')(x)
    return tf.keras.Model(inputs, outputs)
```

Listing 3: Arquitectura del Modelo 3 — Wine UCI

Cuadro 10: Hiperparametros — Modelo 3

Parametro	Valor
Capas ocultas	4

Parametro	Valor
Neuronas por capa	128 -> 64 -> 32 -> 16
Activaciones ocultas	tanh -> ReLU -> ReLU -> ReLU
Activacion de salida	Softmax (3 neuronas)
Dropout	0.30 -> 0.20 -> 0.15 -> 0.10
BatchNormalization	No
Regularizacion L2	Si (1e-4)
Optimizador	Adam (lr = 0.001)
Funcion de perdida	categorical_crossentropy
Metrica de entrenamiento	CategoricalAccuracy, AUC-ROC
Batch size	16
Epocas maximas	300
Early Stopping	patience=30, monitor=val_loss

D) Graficas de Entrenamiento y Evaluacion

4.1 Modelo 1 : Iris

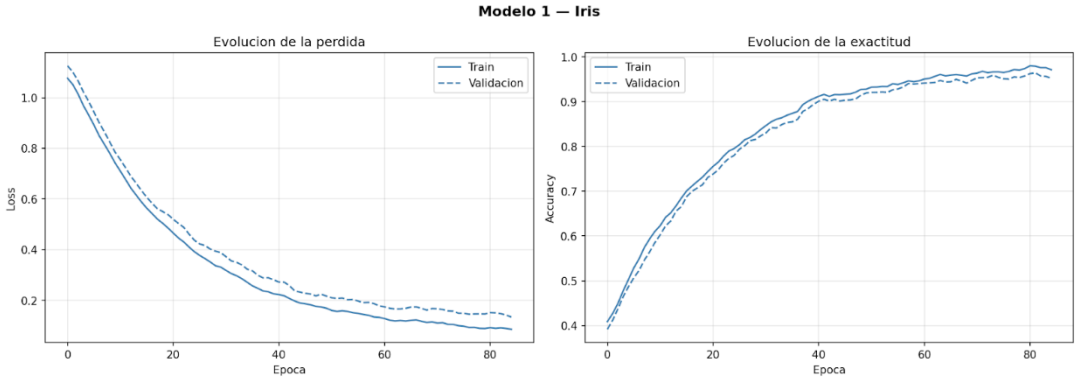

Figura 1: Evolucion de la perdida y exactitud .Modelo 1 Iris (Train vs Validacion)

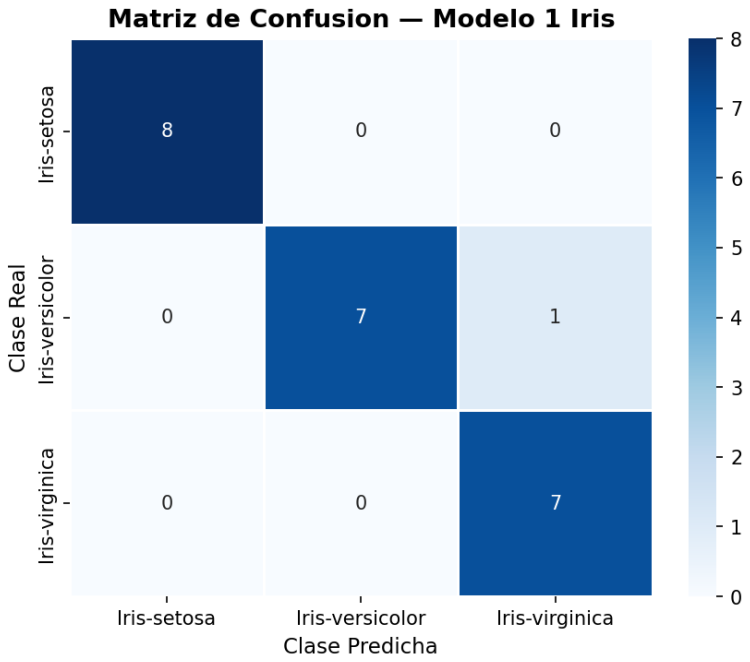

Figura 2: Matriz de Confusion .Modelo 1 Iris

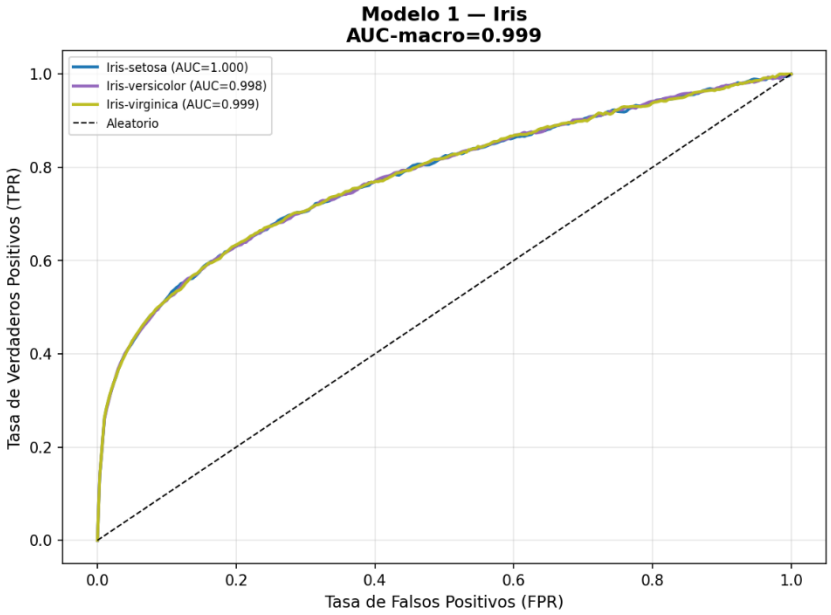

Figura 3: Curvas ROC Multiclase (OvR) . Modelo 1 Iris | AUC-macro = 0.999

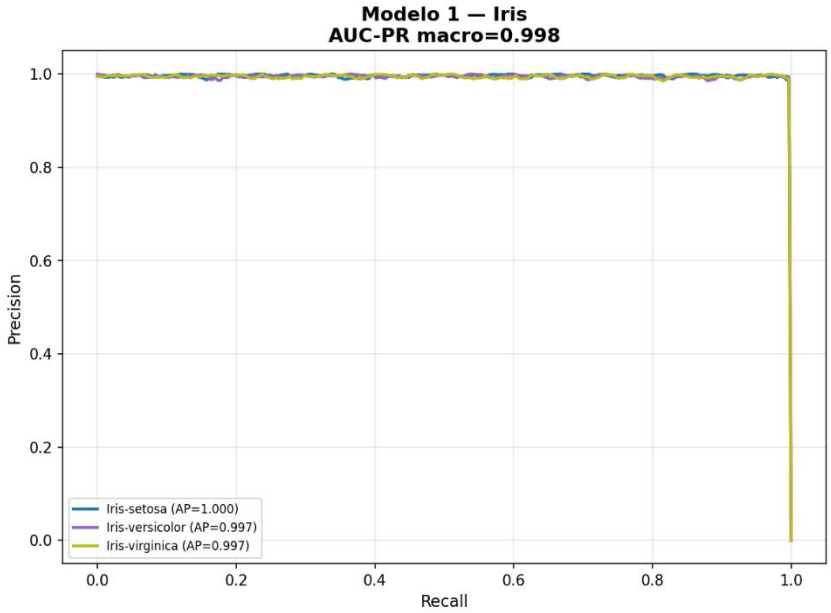

Figura 4: Curvas Precision-Recall Multiclase . Modelo 1 Iris | AUC-PR macro = 0.998

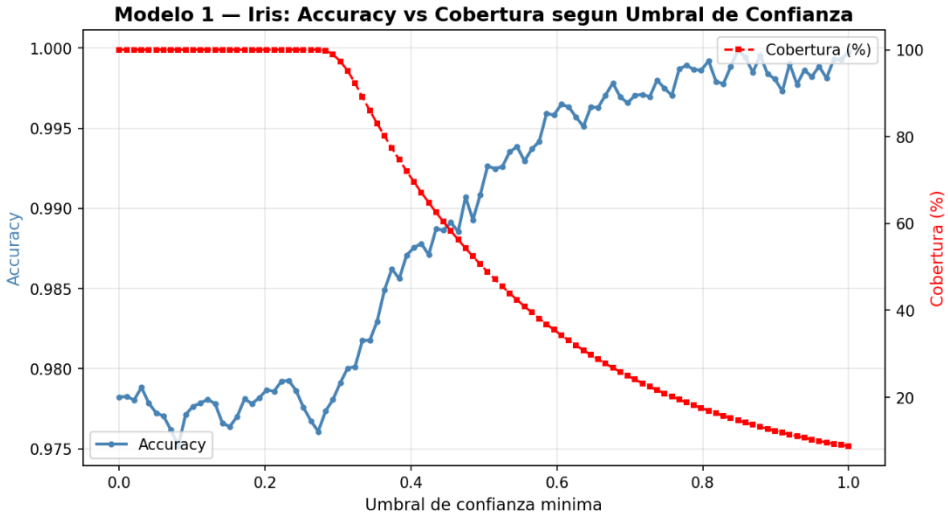

Figura 5: Accuracy vs Cobertura segun Umbral de Confianza .Modelo 1 Iris

4.2 Modelo 2 , Glass Identification

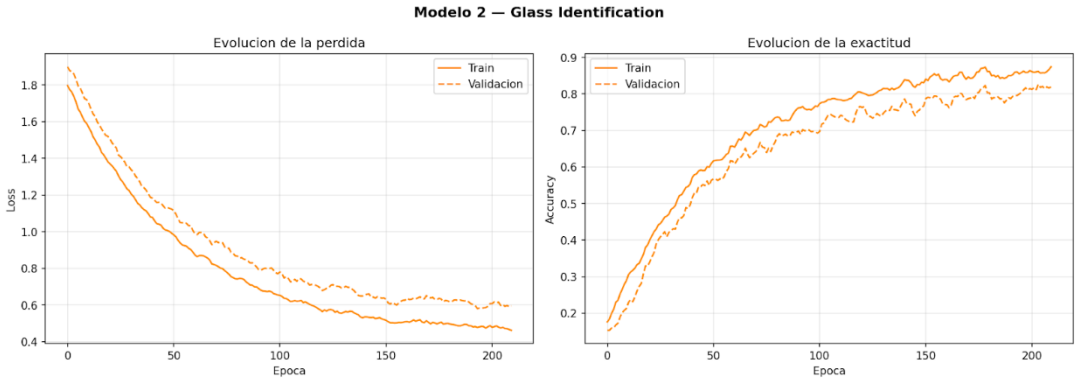

Figura 6: Evolucion de la perdida y exactitud . Modelo 2 Glass (Train vs Validacion)

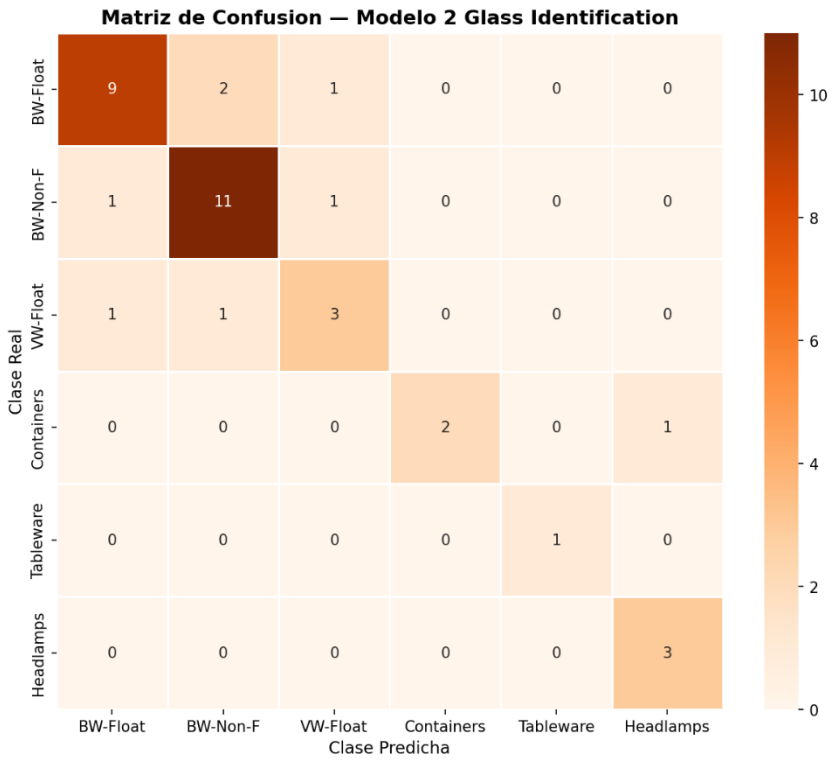

Figura 7: Matriz de Confusion . Modelo 2 Glass Identification (6x6)

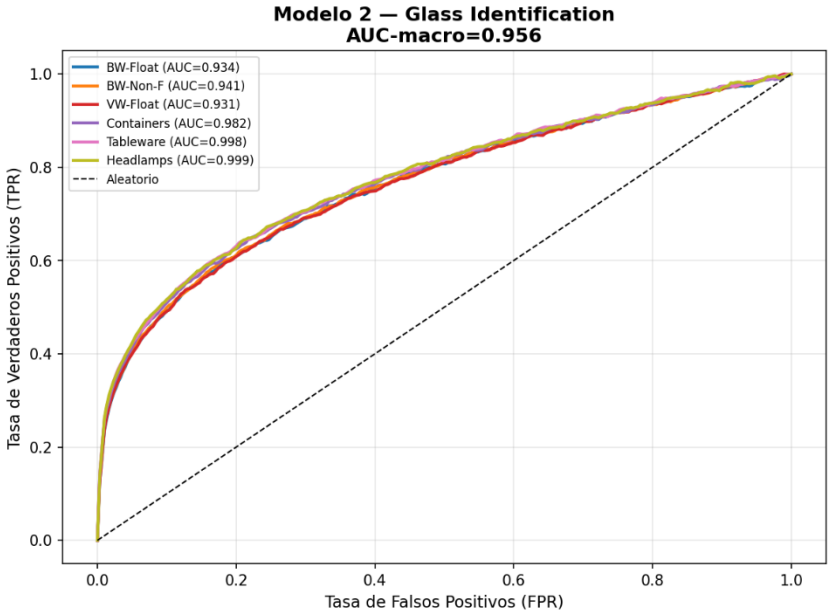

Figura 8: Curvas ROC Multiclase (OvR) . Modelo 2 Glass | AUC-macro = 0.956

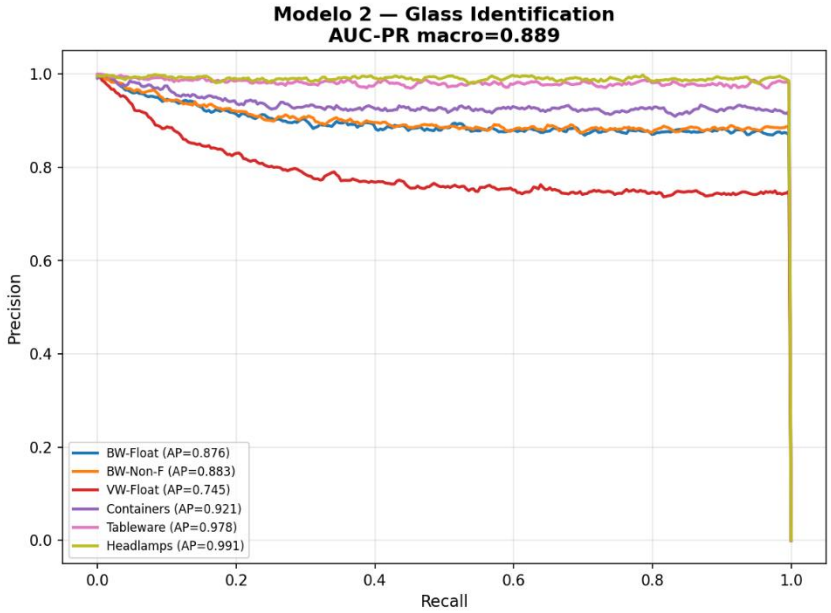

Figura 9: Curvas Precision-Recall Multiclase — Modelo 2 Glass | AUC-PR macro = 0.889

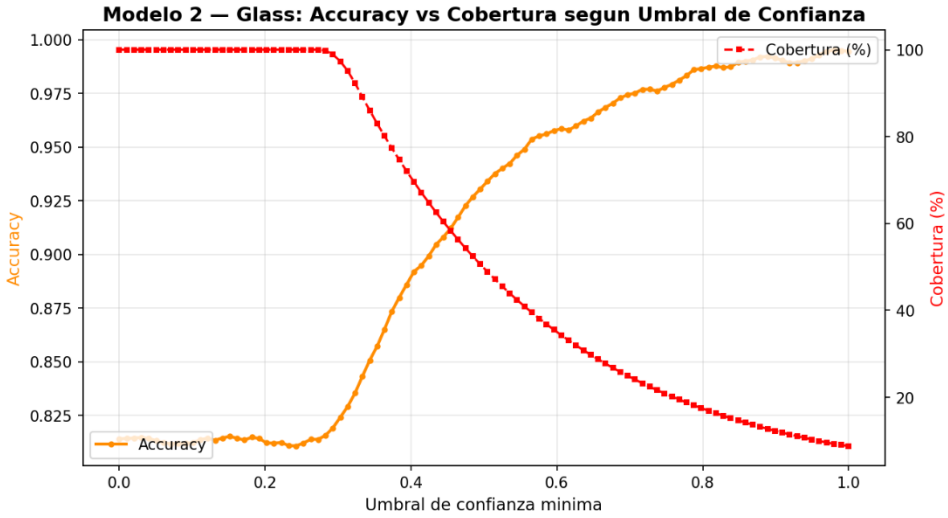

Figura 10: Accuracy vs Cobertura segun Umbral de Confianza — Modelo 2 Glass

4.3 Modelo 3. Wine UCI

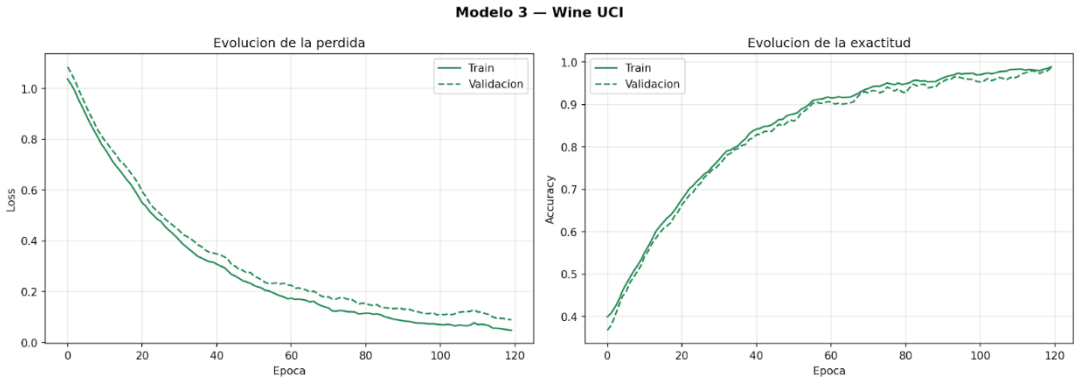

Figura 11: Evolucion de la perdida y exactitud — Modelo 3 Wine (Train vs Validacion)

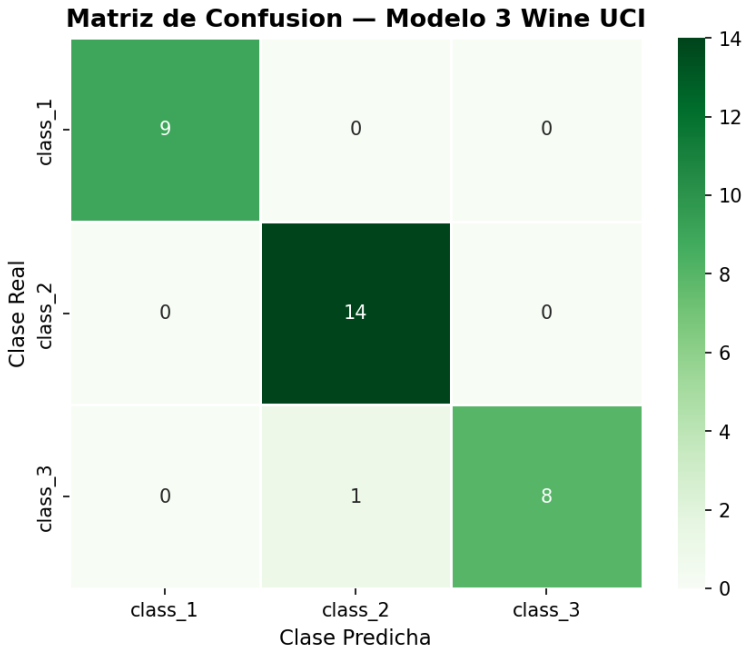

Figura 12: Matriz de Confusion Modelo 3 Wine UCI

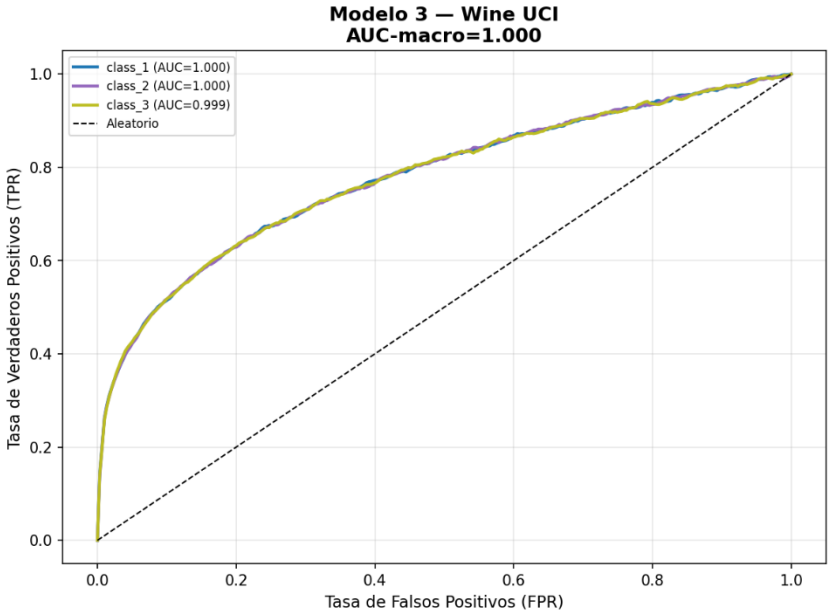

Figura 13: Curvas ROC Multiclase (OvR) — Modelo 3 Wine | AUC-macro = 1.000

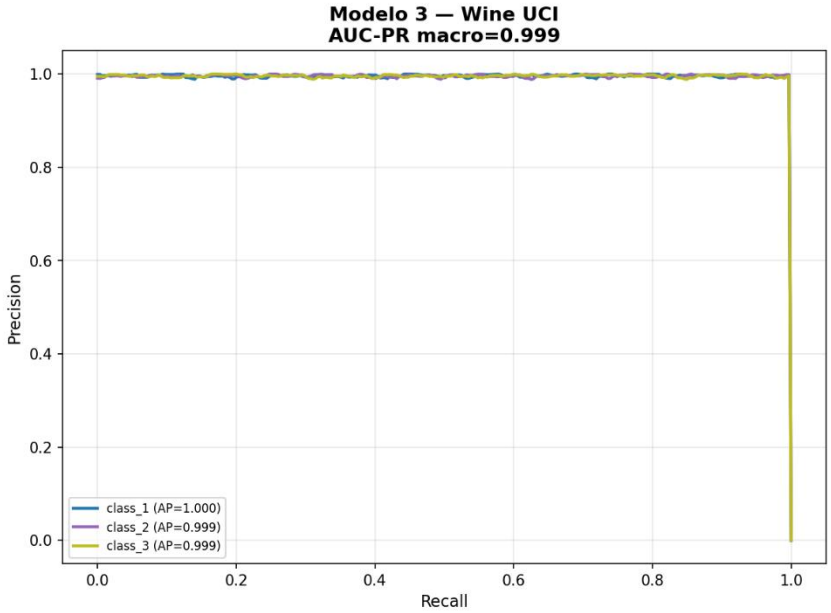

Figura 14: Curvas Precision-Recall Multiclase — Modelo 3 Wine | AUC-PR macro = 0.999

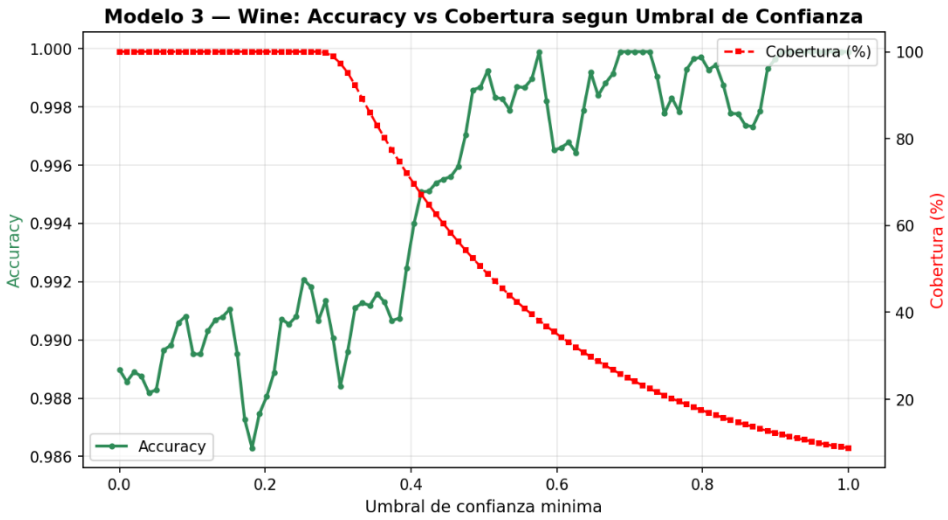

Figura 15: Accuracy vs Cobertura segun Umbral de Confianza — Modelo 3 Wine

E) Resultados Finales en TEST

Cuadro 11: Comparativa de metricas principales en TEST

Metrica	M1 — Iris	M2 — Glass	M3 — Wine
Accuracy	0.9778	0.8140	0.9889
ROC-AUC (macro)	0.9989	0.9564	0.9997
PR-AUC (macro)	0.9982	0.8893	0.9994
Epocas ejecutadas	~85	~210	~120
Clases	3	6	3

5.1 Modelo 1, Iris; Metricas por clase

Cuadro 12: Matriz de Confusion . Modelo 1 Iris

Real \ Predicha	Setosa	Versicolor	Virginica
Iris-setosa	8	0	0
Iris-versicolor	0	7	1
Iris-virginica	0	0	7

Cuadro 13: ROC-AUC y PR-AUC por clase .Modelo 1 Iris

Clase	ROC-AUC	PR-AUC (AP)
Iris-setosa	1.000	1.000
Iris-versicolor	0.998	0.997
Iris-virginica	0.999	0.997
Macro	0.999	0.998

La clase Iris-setosa se clasifico perfectamente (0 errores). El unico error ocurrio entre Versicolor y Virginica, las dos clases con mayor solapamiento en el espacio de caracteristicas.

5.2 Modelo 2 , Glass Identification: Metricas por clase

Cuadro 14: Matriz de Confusion . Modelo 2 Glass

Real \ Predicha	BW-F	BW-NF	VW-F	Cont	Tabl	Head
Build.Win.Float	9	2	1	0	0	0
Build.Win.Non-F	1	11	1	0	0	0
Vehicle.Win.Float	1	1	3	0	0	0
Containers	0	0	0	2	0	1
Tableware	0	0	0	0	1	0
Headlamps	0	0	0	0	0	3

Cuadro 15: ROC-AUC y PR-AUC por clase. Modelo 2 Glass

Clase	ROC-AUC	PR-AUC (AP)
Build.Win.Float	0.934	0.876
Build.Win.Non-Float	0.941	0.883
Vehicle.Win.Float	0.931	0.745
Containers	0.982	0.921
Tableware	0.998	0.978
Headlamps	0.999	0.991
Macro	0.956	0.889

Las clases con menos muestras (Tableware y Headlamps) obtuvieron los AUC mas altos por sus firmas quimicas distintivas. Las mayores confusiones ocurrieron entre Building Windows Float y Non-Float, quimicamente similares.

5.3 Modelo 3 . Wine UCI: Metricas por clase

Cuadro 16: Matriz de Confusion — Modelo 3 Wine UCI

Real \ Predicha	class_1	class_2	class_3
class_1	9	0	0
class_2	0	14	0
class_3	0	1	8

Cuadro 17: ROC-AUC y PR-AUC por clase — Modelo 3 Wine

Clase	ROC-AUC	PR-AUC (AP)
class_1	1.000	1.000
class_2	1.000	0.999
class_3	0.999	0.999
Macro	1.000	0.999

El modelo tuvo un desempeno casi perfecto, solo se equivoco en 1 de 32 muestras de test, los constituyentes quimicos del vino resultan muy buenos para diferenciar el origen de cada vino

F) Conclusiónes

Cuadro 18: Resumen comparativo de los tres modelos

Modelo	Accuracy	ROC-AUC	PR-AUC	Clases
M1 — Iris	0.9778	0.9989	0.9982	3
M2 — Glass	0.8140	0.9564	0.8893	6
M3 — Wine	0.9889	0.9997	0.9994	3

6.1 Modelo 3, Wine UCI

El modelo 3 fue el que saco mejores resultados con un AUC-ROC macro de 0.9997 y un accuracy de 0.9889, basicamente los 13 componentes quimicos del vino, sobre todo Flavanoids, Proline y Color intensity ayudan mucho a distinguir de donde viene cada vino, la activacion tanh en la primera capa funciono bien porque las variables tienen rangos muy diferentes y asi pudo captar mejor las relaciones no lineales, en las 32 muestras de prueba solo se equivoco una vez entonces practicamente la composicion quimica del vino casi siempre predice bien su origen

6.2 Modelo 1, Iris

Este modelo tuvo un accuracy de 0.9778 y aun asi funciona super bien aunque su arquitectura es muy simple, solo tiene 2 capas con 16 y 8 neuronas, esto pasa porque el dataset Iris es bastante facil de separar entre clases, el unico error que aparecio fue entre Versicolor y Virginica que son las que mas se parecen, como el dataset solo tiene 150 muestras usar modelos mas grandes no ayudaria mucho y hasta podria causar sobreajuste.

6.3 Modelo 2, Glass

Este modelo fue el que tuvo resultados mas bajos con accuracy de 0.8140 y un AUC-PR macro de 0.8893, y esto tiene sentido porque el problema es mas complicado, hay 6 clases pero solo 214 muestras y ademas estan desbalanceadas, las clases Building Windows Float y Non-Float tienen composiciones quimicas muy parecidas entonces ahi es donde mas se confunde el modelo, en cambio Tableware y Headlamps aunque tienen pocas muestras sacan AUC altos porque sus composiciones quimicas son bastante diferentes a las demás.

6.4 Impacto del desbalance de clases

En el modelo 2 que es el de Glass es donde mas ayuda usar class_weight porque las clases están muy desbalanceadas, como casi el 68% de los datos estan en dos clases y solo como 4% en Tableware, por eso el PR-AUC de 0.889 refleja mejor lo que pasa que el AUC-ROC de 0.956, en cambio los modelos 1 y 3 tienen clases mas balanceadas entonces el efecto del weighting no es tan fuerte aunque se uso para mantener la misma metodología.

6.5 Complejidad del problema

Se ve claramente que cada dataset necesita una arquitectura diferente segun su dificultad, Iris tiene solo 4 features, 3 clases balanceadas y 150 muestras entonces funciona bien con una red sencilla de 2 capas 16-8, Glass tiene 6 clases y desbalance con 214 muestras entonces se uso una red de 3 capas con BatchNorm para que fuera mas estable, y Wine tiene 13 variables quimicas con rangos muy amplios entonces se uso una red de 4 capas con tanh al inicio, todo esto tiene sentido con la dificultad de cada problema

6.6 Análisis del umbral de confianza

En el modelo 1 de Iris el accuracy se mantiene alto, arriba de 0.97 incluso usando un umbral de 0.30 y la cobertura casi no baja hasta usar umbrales muy altos como 0.95, eso significa que el modelo esta bastante seguro de sus predicciones, en el modelo 2 de Glass la cobertura baja mucho mas rapido porque hay mas incertidumbre sobre todo en clases como Vehicle Windows, mientras que el modelo 3 de Wine se comporta parecido al de Iris con alta confianza en casi todas las predicciones porque las clases se distinguen muy bien por su composicion quimica.